```
#include <stdio.h>
#include <stdlib.h>

Struct Node
{
   Int key;
   Struct Node *left;
   Struct Node *right;
   Int height;
};

Int max(int a, int b)
{
   Return (a > b) ? a : b;
}

Int height(struct Node *N)
{
   If (N == NULL)
      Return 0;

   Return N->height;
}

Struct Node *newNode(int key)
{
   Struct Node *node = (struct Node *)malloc(sizeof(struct Node));
```

```
    Node->key = key;
    Node->left = NULL;
    Node->right = NULL;
    Node->height = 1;

    Return node;
}

Struct Node *rightRotate(struct Node *y)
{
    Struct Node *x = y->left;
    Struct Node *T2 = x->right;

    x->right = y;
    y->left = T2;

    y->height = max(height(y->left), height(y->right)) + 1;
    x->height = max(height(x->left), height(x->right)) + 1;

    return x;
}

Struct Node *leftRotate(struct Node *x)
{
    Struct Node *y = x->right;
    Struct Node *T2 = y->left;
```

```
    y->left = x;
    x->right = T2;

    x->height = max(height(x->left), height(x->right)) + 1;
    y->height = max(height(y->left), height(y->right)) + 1;

    return y;
}

Int getBalance(struct Node *N)
{
    If (N == NULL)
        Return 0;

    Return height(N->left) – height(N->right);
}

Struct Node *insertNode(struct Node *node, int key)
{
    If (node == NULL)
        Return newNode(key);

    If (key < node->key)
        Node->left = insertNode(node->left, key);
    Else if (key > node->key)
        Node->right = insertNode(node->right, key);
```

```
    Else
        Return node;

    Node->height = 1 + max(height(node->left), height(node->right));

    Int balance = getBalance(node);

    If (balance > 1 && key < node->left->key)
        Return rightRotate(node);

    If (balance < -1 && key > node->right->key)
        Return leftRotate(node);

    If (balance > 1 && key > node->left->key)
    {
        Node->left = leftRotate(node->left);
        Return rightRotate(node);
    }

    If (balance < -1 && key < node->right->key)
    {
        Node->right = rightRotate(node->right);
        Return leftRotate(node);
    }

    Return node;
}
```

```
Struct Node *minValueNode(struct Node *node)
{
   Struct Node *current = node;

   While (current->left != NULL)
      Current = current->left;

   Return current;
}

Struct Node *deleteNode(struct Node *root, int key)
{
   If (root == NULL)
      Return root;

   If (key < root->key)
      Root->left = deleteNode(root->left, key);

   Else if (key > root->key)
      Root->right = deleteNode(root->right, key);

   Else
   {
      If ((root->left == NULL) || (root->right == NULL))
      {
         Struct Node *temp = root->left ?
```

```
                    Root->left : root->right;

        If (temp == NULL)
        {
            Temp = root;
            Root = NULL;
        }
        Else
        {
            *root = *temp;
        }

        Free(temp);
    }
    Else
    {
        Struct Node *temp = minValueNode(root->right);

        Root->key = temp->key;
        Root->right = deleteNode(root->right, temp->key);
    }
}

If (root == NULL)
    Return root;

Root->height = 1 + max(height(root->left),
```

```
                    Height(root->right));

    Int balance = getBalance(root);

    If (balance > 1 && getBalance(root->left) >= 0)
        Return rightRotate(root);

    If (balance > 1 && getBalance(root->left) < 0)
    {
        Root->left = leftRotate(root->left);
        Return rightRotate(root);
    }

    If (balance < -1 && getBalance(root->right) <= 0)
        Return leftRotate(root);

    If (balance < -1 && getBalance(root->right) > 0)
    {
        Root->right = rightRotate(root->right);
        Return leftRotate(root);
    }

    Return root;
}

Void printPreOrder(struct Node *root)
{
```

```
    If (root != NULL)
    {
        Printf("%d ", root->key);
        printPreOrder(root->left);
        printPreOrder(root->right);
    }
}

Int main()
{
    Struct Node *root = NULL;

    Root = insertNode(root, 9);
    Root = insertNode(root, 5);
    Root = insertNode(root, 10);
    Root = insertNode(root, 0);
    Root = insertNode(root, 6);
    Root = insertNode(root, 11);
    Root = insertNode(root, -1);
    Root = insertNode(root, 1);
    Root = insertNode(root, 2);

    Printf("Preorder traversal of the constructed AVL tree is \n");
    printPreOrder(root);

    root = deleteNode(root, 10);
```

```
    printf("\n\nPreorder traversal after deletion of 10 \n");
    printPreOrder(root);

    return 0;
}
```